ASE 370C - Supplemental Information for Real-Time Autonomous Target Trajectory Following

Akhil Gharpurey

April 2026

Controller Architecture

Figure 1 shows the nested loop controller design for this project. We elected to choose a nested loop controller to discretize the continuous target motion into steps, which the inner control loop can easily track using fundamental PID control techniques. Consequently, the controller will drive the ownship aircraft to the reference parameter over a series of timesteps governed by the sampling rate of the outer loop.

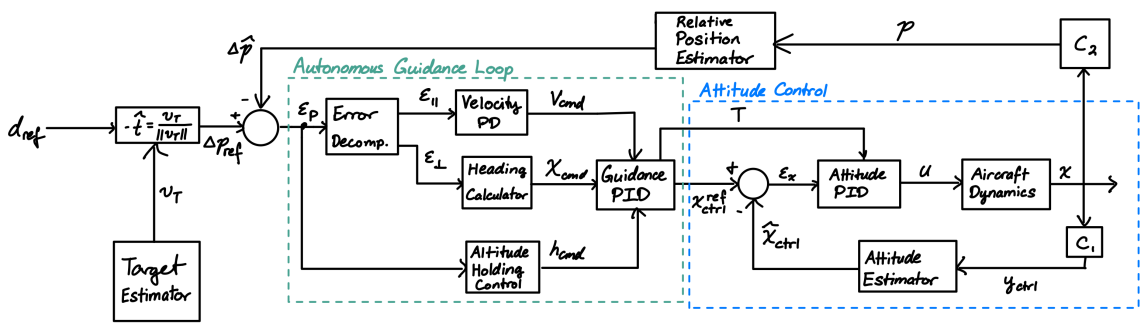

Figure 1: Nested Loop Controller Architecture for Real-Time Autonomous Target Pursuit.

2 Hz Outer Guidance Loop

The outer guidance loop takes in the desired reference and translates this value to attitude commands. The external reference signal in the control loop is the reference standoff distance, d_{ref}. The controller derives an estimate of the target velocity and normalizes it to obtain the target direction, $\hat{t}$. The standoff distance is then vectorized by simply scaling $\hat{t}$ by $-d_{ref}$. The tracking error between the desired standoff displacement vector, Δp_{ref} and the observed standoff displacement vector, $\Delta\hat{p}$. This error vector, ε_p, is subsequently decomposed into a parallel (on-track) component, an orthogonal (cross-track) component, and an altitude component. The on-track and cross-track components are derived using an orthogonal projection, shown in Equation 1. The altitude error is simply taken as the z-component of ε_p, as the altitude error in the desired state is zero.

$$\varepsilon_{||} = \hat{t}^\top \varepsilon_p \qquad\qquad \varepsilon_\perp = ||\varepsilon_p - \hat{t}\varepsilon_{||}|| \tag{1}$$

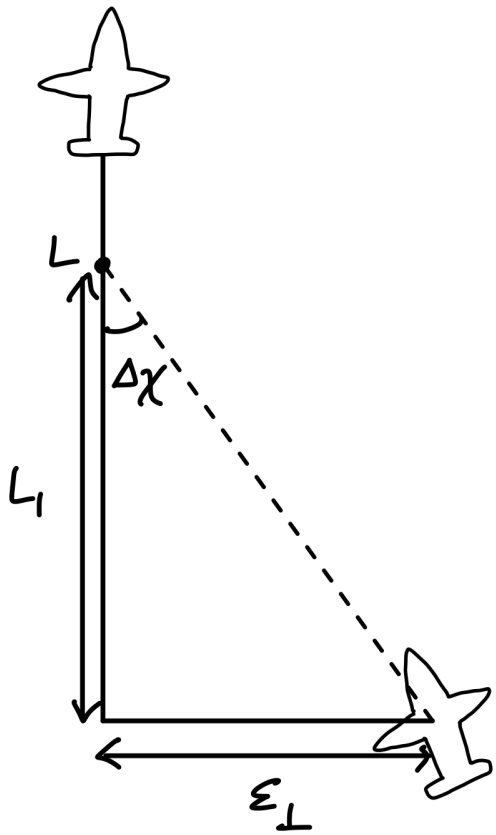

Figure 2: Lookahead Approach for Heading Calculation.

These error components are then utilized to derived commands for the ownship velocity, heading, and altitude. We use a PD controller to acquire the desired ownship velocity as shown in Equation 2. This controller was selected to minimize risk of overshoot and subsequent oscillations so that the throttle is able to drive the aircraft towards reference velocities with ease.

$$V_{cmd} = V_T + K_{p||}\varepsilon_{||} + K_{d||}\dot{\varepsilon}_{||} \tag{2}$$

The heading calculation is derived using a lookahead approach. Using a prescribed lookahead distance, the controller determines a heading angle differential using the trigonometric relationship shown in Figure 2. The heading command is therefore derived as shown in Equation **??**. Note that k_y is an additional parameter to tune aggressiveness of derived heading command. However, for geometric accuracy, this is set to 1.

$$\chi_{cmd} = \chi_{tgt} + k_y \arctan\left(\frac{\varepsilon_\perp}{L_1}\right) \tag{3}$$

Lastly, the altitude command is simply the sum of the current ownship altitude and the altitude error, ε_h. Using these commands, we derive commands for roll, pitch, and throttle as shown below. The desired roll φ_{cmd} is derived from the formula for coordinated turn in a fixed-wing UAV, where

the heading error derivative is approximated by $\dot{\varepsilon}_\chi \approx K_{d\chi}\varepsilon_\chi$. The pitch angle is determined by a PID controller with anti-windup clamp measures built in to prevent derivation of control inputs beyond physical limits. The throttle command is derived using a PI controller from the velocity command.

$$\begin{aligned}
\varphi_{cmd} &= \arctan\left(\frac{V_a K_{d\chi}\varepsilon_\chi}{g}\right) \\
\theta_{cmd} &= \text{sat}\left(K_{ph}\varepsilon_h + K_{Ih}\int_0^t \varepsilon_h(\tau)d\tau + K_{dh}\dot{\varepsilon}_h\right) \\
T_{cmd} &= T_0 + K_{pV}\varepsilon_V + K_{IV}\int_0^t \varepsilon_V(\tau)d\tau
\end{aligned}$$

Therefore, the reference vector fed into the inner attitude loop is defined by $x_{ctrl}^{ref} = \begin{bmatrix}\varphi_{cmd} & \theta_{cmd} & \beta_{cmd}\end{bmatrix}^\top$. We augment the roll and pitch with an additional state, β_{cmd}, denoting sideslip in the aircraft. Since we want to maintain the aircraft trajectory, $\beta_{cmd} = 0$ always.

200 Hz Inner Attitude Control Loop

The inner loop governs the aircraft attitude over time based on the reference attitude vector, x_{ctrl}^{ref}. The attitude control is a multi-input multi-output (MIMO) PID anti-windup controller which yields the desired aileron, rudder, and elevator angles, which are subsequently fed into the aircraft dynamics along with the previously derived throttle command. This is mathematically formulated in the equations below.

$$\begin{aligned}
\text{Roll} \rightarrow \text{Aileron: } \delta_a &= \text{sat}\left(K_{p\varphi}\varepsilon_\varphi + K_{I\varphi}\int_0^t \varepsilon_\varphi(\tau)d\tau + K_{d\varphi}\dot{\varepsilon}_\varphi\right) \\
\text{Sideslip} \rightarrow \text{Rudder: } \delta_r &= \text{sat}\left(K_{p\beta}\varepsilon_\beta + K_{I\beta}\int_0^t \varepsilon_\beta(\tau)d\tau + K_{d\beta}\dot{\varepsilon}_\beta\right) \\
\text{Pitch} \rightarrow \text{Elevator: } \delta_e &= \text{sat}\left(K_{p\theta}\varepsilon_\theta + K_{I\theta}\int_0^t \varepsilon_\theta(\tau)d\tau + K_{d\theta}\dot{\varepsilon}_\theta\right)
\end{aligned}$$

The aircraft dynamics are simply propagated through the high-fidelity simulation environment, XPlane 12. On the inner feedback path, we derive the roll, pitch, and sideslip and compute the error between these parameters and the desired reference values provided by the guidance loop. On the outer feedback path, the error between the ownship and target aircraft positions is derived using simple vector subtraction.

It is important to note that the outer loop is sampled at a frequency of 2 Hz, and the inner loop is sampled at a frequency of 200 Hz. Therefore, within each outer loop iteration, 100 inner loop iterations are conducted. This consequently treats each reference command generated by the outer loop as a step input for which the attitude PID control loop tracks. Therefore, we are able to achieve real-time performance while minimizing sporadic behavior in the control response.

On Usage of Generative AI

In this project, generative AI tools such as ChatGPT 5.2 and Claude Opus were used to accelerate the progress of the project. In the initial stages of the project, we formulated the nested loop concept organically as well as the inner attitude loop, as these are commonly known configurations that we were familiar with. However, in designing the architecture, we were unfamiliar with methods to map the reference standoff distance to the attitude reference commands. We therefore first took inspiration from literature such as [1] in which they designed a waypoint guidance mission. This paper provided clear visualization on outer loop guidance implementation through the Simulink user interface, but the underlying mathematical principles were still unclear. We therefore utilized ChatGPT as an interpolation mechanism between the problem statement for our project and the waypoint heuristic mission used in [1]. From continued interactions with ChatGPT, we eventually landed on the autonomous guidance loop visualized in Figure 1.

Claude Opus 4.6 was additionally leveraged in developing simulations and implementation methods. After formalizing the controller architecture shown in Figure 1, we uploaded the visualization to Claude Opus 4.6 and asked it to develop a simulation in MATLAB for the target following controller as well as a mathematical fixed-wing UAV model with prescribed mass and inertial parameters of 25 kg and $\approx 5 \times 10^{-3}$ kg $\cdot$ m^2 along the $x, y,$ and z axes. These values were based on model fixed-wing UAVs we were initially familiar with and not necessarily mass parameters for existing industry aircrafts. It is important to note that Claude Opus 4.6 provided us with initial PID gains for all control blocks. Therefore, we tuned these constants ourselves to achieve the responses shown in the video presentation.

After completing the PID gain optimization for the low-fidelity simulation, we then asked Claude Opus 4.6 to translate this simulation into an XPlane Python plugin for high-fidelity simulation deployment. After Claude generated this plugin code, our interactions with generative AI effectively ended here. There were some notable errors in the translation process that we needed to fix. One of these was the flipped sign convention between XPlane and MATLAB. Additionally, we needed to retune the PID constants, as the Cirrus SR-22 model parameters deviate heavily from our low-fidelity fictitious aircraft model parameters. After completing this process, we effectively completed the project as a whole, with generative AI being a large accelerator for code production and deployment as well as a source of knowledge beyond the scope of an undergraduate feedback control systems course.

References

[1] N. Ruenruedeepan, Y. Kanokmedhakul, A. Nonut, P. Champasak, S. Bureerat, and N. Pholdee, "Design and development of gnc systems for fully autonomous flight of a small fixed-wing uavs flight simulator using a meta-heuristic," 05 2021.